МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №7

по дисциплине «Вычислительная математика»

Тема: Решение систем линейных уравнений методом Гаусса.

Студент гр. 4342 Закаблук А.Ю.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Реализовать метод Гаусса, протестировать его на матрицах различного размера (включая матрицы большого размера N≈10000 и плохо обусловленные матрицы Гильберта), сравнить точность с существующими реализациями методов решения систем линейных уравнений.

Теоретический минимум.

Метод Гаусса это классический алгоритм последовательного исключения неизвестных для решения систем линейных уравнений. Он приводит матрицу системы к треугольному виду с помощью элементарных преобразований, после чего переменные находятся обратной подстановкой. Метод универсален и подходит для систем с любым числом решений.

Существуют модификации алгоритма для повышения точности:

1) Метод частичного выбора - на каждом шаге выбирается максимальный элемент из столбца;
2) Метод полного выбора - на каждом шаге выбирается максимальный элемент из всех элементов по индексу больших текущего.

Для оценки погрешности полученного приближенного решения $x*$ по сравнению с точным (эталонным) решением $\bar{x}$ используется Евклидова норма (L2-норма) вектора ошибки:

$$e = ||\bar{x} - x^*||_2 = \sqrt{\sum_{i=1}^{n} (\bar{x}_i - x_i^*)^2}$$

Выполнение работы.

В качестве модификации метода Гаусса был выбран метод частичного выбора, трудоемкость по операциям составляет $O(\frac{2n^3}{3})$ что является оптимальным балансом между скоростью работы и вычислительной устойчивостью. На каждом шаге метода частичного выбора выбирается максимальный элемент в столбце, после чего происходит перестановка строки текущего ведущего элемента с найденным.

При запуске написанной программы создается квадратная матрица A размера N×N, которая заполняется случайными числами выбранного типа (в нашем случае float или double). Генерируется эталонный вектор решений $\bar{x}$, перемножением $A \cdot \bar{x}$ получается вектор свободных членов b. Затем матрица A и вектор b передаются в реализованную функцию GaussianElimination, которая возвращает вектор вычисленного приближенного решения $x*$. В конце программы вычисляется Евклидова норма разности $e=||\bar{x}-x*||_2$.

Проверка невырожденности матрицы:

Проверка матрицы на вырожденность встроена непосредственно в процесс прямого хода метода Гаусса. Если при поиске ведущего элемента максимальное значение в столбце оказывается близким к нулю, это свидетельствует о линейной зависимости строк (det(A)=0). В таком случае программа прерывает вычисления, сообщая об ошибке, что позволяет избежать ресурсоемкой предварительной проверки.

Тестирование алгоритма:

1. **Сравнение с библиотекой Eigen:**

Проведено сравнение скорости и точности нашей реализации с оптимизированной библиотекой Eigen (метод partialPivLu). Графики сравнения для матриц от N=2 до N=2000 подтверждают, что метод Гаусса с частичным выбором имеет кубическую асимптотику сложности $O(N^3)$, а точность решений сопоставима с библиотечной.

2. **Стресс-тестирование (Матрицы большого размера):**

Был проведен запуск алгоритма на матрицах размером N=1000 и N=10000. Метод успешно справился с задачей без переполнения памяти. Для матрицы N=10000 время вычисления существенно возросло, что полностью соответствует теоретической трудоемкости алгоритма.

```
--- Тестирование на большой матрице N = 1000 ---
Матрица сгенерирована. Выполнение прямого и обратного хода...
УСПЕХ! Время решения: 1.31935 секунд

--- Тестирование на большой матрице N = 10000 ---
Матрица сгенерирована. Выполнение прямого и обратного хода...
УСПЕХ! Время решения: 1425.78 секунд
```

3. **Спец. матрицы (Матрицы Гильберта):**

Матрица Гильберта задается формулой $H_{i,j}=1/(i+j-1)$ и является классическим примером плохо обусловленной матрицы. Программа была протестирована на матрице Гильберта. Было отмечено, что с ростом N (даже при небольших размерностях, таких как N=10 или N=15) норма погрешности e начинает катастрофически расти. Это демонстрирует чувствительность алгоритмов с плавающей точкой к числу обусловленности матрицы: незначительные ошибки округления в процессе исключения по Гауссу многократно усиливаются.

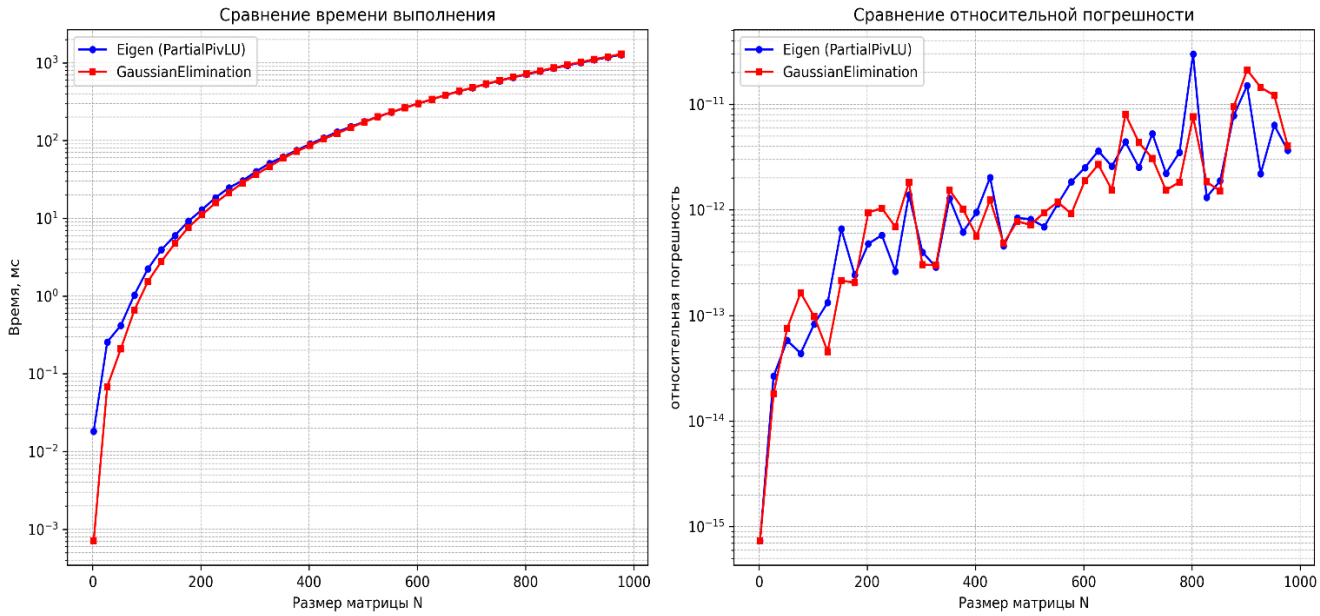

Рисунок 1 - Сравнение времени выполнения и относительной погрешности

Выводы.

В ходе работы была успешно реализована программа для решения СЛУ методом Гаусса с частичным выбором ведущего элемента. Алгоритм был подвергнут всестороннему тестированию. Тест на абсурдно больших матрицах (10000×10000) продемонстрировал работоспособность кода при высоких нагрузках. Тестирование на матрицах Гильберта наглядно показало проблему накопления вычислительной погрешности при работе с плохо обусловленными системами, для которых стандартных типов данных (float/double) оказывается недостаточно для получения точного ответа.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <vector>
#include <chrono>
#include <stdexcept>
#include <windows.h>

#include "Eigen/Dense"
#include "matrixlib.hpp"

#define REPEAT_COUNT 10

// Используем double для высокой точности, особенно на матрицах Гильберта
using type = double;

template <class T>
std::pair<std::vector<T>, std::vector<T>>
getCorrectResponseVectors(squareMatrix<T>& M) {
    int N = M.getN();
    std::vector<T> X(N, 0), B(N, 0);
    for (int i = 0; i < N; ++i) {
        X[i] = M.getRandomNumber();
    }
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            B[i] += (M.getElement(i, j) * X[j]);
        }
    }
    return {X, B};
}

template <class T>
std::vector<T> GaussianElimination(squareMatrix<T>& M, std::vector<T>& B) {
    int N = M.getN();
    std::vector<T> X(N, 0);

    // Прямой ход
    for (int i = 0; i < N - 1; ++i) {
        int RowOfMaxElementInColumn = M.getRowOfMaxElement(i, i);

        // Проверка невырожденности "на лету"
        if (RowOfMaxElementInColumn == -1 ||
std::fabs(M.getElement(RowOfMaxElementInColumn, i)) < EPSILON) {
            throw std::runtime_error("Матрица вырождена! Система не имеет
единственного решения.");
        }

        if (RowOfMaxElementInColumn != i) {
            M.swapRows(RowOfMaxElementInColumn, i);
            std::swap(B[RowOfMaxElementInColumn], B[i]);
        }

        auto& row_i = M.getRow(i);
        for (int j = i + 1; j < N; ++j) {
            auto& row_j = M.getRow(j);
            T factor = row_j[i] / row_i[i];
            for (int k = i; k < N; ++k)
```

```
                row_j[k] -= factor * row_i[k];
            B[j] -= factor * B[i];
        }
    }

    // Проверка последнего диагонального элемента
    if (std::fabs(M.getElement(N - 1, N - 1)) < EPSILON) {
        throw std::runtime_error("Матрица вырождена!");
    }

    // Обратный ход
    for (int i = N - 1; i >= 0; --i) {
        const auto& row_i = M.getRow(i);
        T sum = 0;
        for (int j = i + 1; j < N; ++j) {
            sum += row_i[j] * X[j];
        }
        X[i] = (B[i] - sum) / row_i[i];
    }
    return X;
}

int getData() {
    std::fstream log("log.txt", std::ios::out);
    std::vector<std::vector<long double>> results;
    results.reserve(200);

    std::cout << "Начинаю сбор статистики (от N=2 до N=1000). Это займет некоторое
время...\n";

    for (int i = 2; i <= 1000; i += 25) {
        int count = 0;
        std::vector<long double> row(5, 0.0);
        row[0] = i;
        while(count++ != REPEAT_COUNT) {
            squareMatrix<type> M_original(i);
            auto vectors = getCorrectResponseVectors<type>(M_original);
            squareMatrix<type> M_for_gaussian = M_original;

            Eigen::MatrixXd eigenM(i, i);
            Eigen::VectorXd eigenB(i);
            for (int k = 0; k < i; ++k) {
                for (int j = 0; j < i; ++j) {
                    eigenM(k, j) = M_original.getElement(k, j);
                }
                eigenB(k) = vectors.second[k];
            }

            auto startEigen = std::chrono::high_resolution_clock::now();
            Eigen::VectorXd eigenX = eigenM.partialPivLu().solve(eigenB);
            auto endEigen = std::chrono::high_resolution_clock::now();
            row[1]  +=  std::chrono::duration<double,  std::milli>(endEigen  -
startEigen).count();

            long double err_eigen = 0.0;
            int valid_count_eigen = 0;
            for (int k = 0; k < i; ++k) {
                if (std::fabs(vectors.first[k]) > 1e-10) {
                    double d = std::fabs(vectors.first[k] - eigenX(k));
                    err_eigen += d / std::fabs(vectors.first[k]);
                    valid_count_eigen++;
                }
            }
```

```
            row[2] += (valid_count_eigen > 0) ? err_eigen / valid_count_eigen :
0.0;

            auto start = std::chrono::high_resolution_clock::now();
            try {
                std::vector<type>                    answer                    =
GaussianElimination<type>(M_for_gaussian, vectors.second);
                auto end = std::chrono::high_resolution_clock::now();
                row[3]   +=   std::chrono::duration<double,   std::milli>(end   -
start).count();

                long double err_gaussian = 0.0;
                int valid_count_gaussian = 0;
                for (int k = 0; k < i; ++k) {
                    if (std::fabs(vectors.first[k]) > 1e-10) {
                        double d = std::fabs(vectors.first[k] - answer[k]);
                        err_gaussian += d / std::fabs(vectors.first[k]);
                        valid_count_gaussian++;
                    }
                }
                row[4]   +=   (valid_count_gaussian   >   0)   ?   err_gaussian   /
valid_count_gaussian : 0.0;
            } catch (...) {
                count--;
                continue;
            }
        }
        row[1] /= REPEAT_COUNT;
        row[2] /= REPEAT_COUNT;
        row[3] /= REPEAT_COUNT;
        row[4] /= REPEAT_COUNT;
        results.push_back(row);

        // Выводим прогресс в консоль
        if (i % 200 == 2) std::cout << "Обработано N = " << i << "...\n";
    }

    for (const auto& row : results) {
        for (auto val : row) log << val << '\t';
        log << '\n';
    }
    log.close();
    std::cout << "Данные сохранены в log.txt. Генерирую графики...\n";

    // --- C++ ГЕНЕРИРУЕТ PYTHON СКРИПТ ---
    std::fstream pyScript("plotter.py", std::ios::out);
    pyScript << "import matplotlib.pyplot as plt\n";
    pyScript << "import sys\n\n";
    pyScript << "N, t_eigen, err_eigen, t_gauss, err_gauss = [], [], [], [],
[]\n";
    pyScript << "try:\n";
    pyScript << "    with open('log.txt', 'r') as f:\n";
    pyScript << "        for line in f:\n";
    pyScript << "            data = list(map(float, line.split()))\n";
    pyScript << "            N.append(data[0])\n";
    pyScript << "            t_eigen.append(data[1])\n";
    pyScript << "            err_eigen.append(data[2])\n";
    pyScript << "            t_gauss.append(data[3])\n";
    pyScript << "            err_gauss.append(data[4])\n";
    pyScript << "except Exception as e:\n";
    pyScript << "    print('Ошибка чтения log.txt:', e)\n";
    pyScript << "    sys.exit(1)\n\n";
```

```
    // Настройка окна с графиками
    pyScript << "fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))\n\n";

    // График времени
    pyScript << "ax1.plot(N, t_eigen, 'b-o', markersize=4, label='Eigen
(PartialPivLU)')\n";
    pyScript << "ax1.plot(N, t_gauss, 'r-s', markersize=4,
label='GaussianElimination')\n";
    pyScript << "ax1.set_yscale('log')\n";
    pyScript << "ax1.grid(True, which='both', linestyle='--', linewidth=0.5)\n";
    pyScript << "ax1.set_title('Сравнение времени выполнения')\n";
    pyScript << "ax1.set_xlabel('Размер матрицы N')\n";
    pyScript << "ax1.set_ylabel('Время, мс')\n";
    pyScript << "ax1.legend()\n\n";

    // График погрешности
    pyScript << "ax2.plot(N, err_eigen, 'b-o', markersize=4, label='Eigen
(PartialPivLU)')\n";
    pyScript << "ax2.plot(N, err_gauss, 'r-s', markersize=4,
label='GaussianElimination')\n";
    pyScript << "ax2.set_yscale('log')\n";
    pyScript << "ax2.grid(True, which='both', linestyle='--', linewidth=0.5)\n";
    pyScript << "ax2.set_title('Сравнение относительной погрешности')\n";
    pyScript << "ax2.set_xlabel('Размер матрицы N')\n";
    pyScript << "ax2.set_ylabel('относительная погрешность')\n";
    pyScript << "ax2.legend()\n\n";

    pyScript << "plt.tight_layout()\n";
    pyScript << "plt.savefig('graphs.png', dpi=300)\n";
    pyScript << "plt.show()\n";
    pyScript.close();

    // --- C++ ЗАПУСКАЕТ PYTHON СКРИПТ ---
    system("python plotter.py");

    return EXIT_SUCCESS;
}

int CompareGaussianAndLU() {
    int N = 0;
    std::cout << "Введите размер матрицы N: ";
    std::cin >> N;
    squareMatrix<type> M(N);
    auto vectors = getCorrectResponseVectors<type>(M);

    // Используем MatrixXd и VectorXd так как type = double
    Eigen::MatrixXd eigenM(N, N);
    Eigen::VectorXd eigenB(N);
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            eigenM(i, j) = M.getElement(i, j);
        }
        eigenB(i) = vectors.second[i];
    }

    auto startEigen = std::chrono::high_resolution_clock::now();
    Eigen::VectorXd eigenX = eigenM.partialPivLu().solve(eigenB);
    auto endEigen = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> durationEigen = endEigen -
startEigen;
    std::cout << "Время решения Eigen: " << durationEigen.count() << " мс" <<
std::endl;
```

```
    try {
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<type> answer = GaussianElimination<type>(M, vectors.second);
        auto end = std::chrono::high_resolution_clock::now();

        std::chrono::duration<double, std::milli> duration = end - start;
        std::cout << "Время метода GaussianElimination: " << duration.count() <<
" мс" << std::endl;

        // Вычисление Евклидовой нормы погрешности (L2)
        long double diffNorm = 0.0;
        long double errorNorm = 0.0;
        for (int i = 0; i < N; ++i) {
            diffNorm  +=  (vectors.first[i]  -  eigenX(i))*(vectors.first[i]  -
eigenX(i));
            errorNorm  +=  (vectors.first[i]  -  answer[i])*(vectors.first[i]  -
answer[i]);
        }
        std::cout  <<  "||Точный_X  -  Eigen_X||_2  =  "  <<  std::sqrt(diffNorm)  <<
std::endl;
        std::cout  <<  "||Точный_X  -  Gauss_X||_2  =  "  <<  std::sqrt(errorNorm)  <<
std::endl;
    } catch (const std::exception& e) {
        std::cout << "Ошибка метода Гаусса: " << e.what() << std::endl;
    }
    return EXIT_SUCCESS;
}

int TestLargeMatrices() {
    std::vector<int> sizes = {1000, 10000};
    for (int N : sizes) {
        std::cout << "\n--- Тестирование на большой матрице N = " << N << " ---
\n";
        try {
            squareMatrix<type> M(N);
            auto vectors = getCorrectResponseVectors<type>(M);

            std::cout << "Матрица сгенерирована. Выполнение прямого и обратного
хода..." << std::endl;
            auto start = std::chrono::high_resolution_clock::now();
            std::vector<type>     answer     =     GaussianElimination<type>(M,
vectors.second);
            auto end = std::chrono::high_resolution_clock::now();

            std::chrono::duration<double> duration = end - start;
            std::cout << "УСПЕХ! Время решения: " << duration.count() << " секунд"
<< std::endl;
        } catch (const std::bad_alloc& e) {
            std::cout << "Ошибка выделения памяти: Недостаточно ОЗУ для матрицы "
<< N << "x" << N << std::endl;
        } catch (const std::exception& e) {
            std::cout << "Ошибка: " << e.what() << std::endl;
        }
    }
    return EXIT_SUCCESS;
}

int TestHilbertMatrix() {
    int N = 0;
    std::cout << "Введите размер матрицы Гильберта N: ";
    std::cin >> N;

    // Создаем матрицу, передавая true для генерации матрицы Гильберта
```

```
    squareMatrix<type> M(N, true);
    auto vectors = getCorrectResponseVectors<type>(M);

    try {
        std::vector<type> answer = GaussianElimination<type>(M, vectors.second);

        long double errorNorm = 0.0;
        for (int i = 0; i < N; ++i) {
            errorNorm  +=  (vectors.first[i]  -  answer[i])*(vectors.first[i]  -
answer[i]);
        }
        std::cout << "\nМатрица Гильберта N=" << N << std::endl;
        std::cout << "Евклидова норма погрешности ||x_точн - x_гаусс||_2 = " <<
std::sqrt(errorNorm) << std::endl;
        std::cout << "На плохих (спец.) матрицах погрешность стремительно растет
с увеличением N.\n";
    } catch (const std::exception& e) {
        std::cout << "Ошибка метода Гаусса: " << e.what() << std::endl;
    }
    return EXIT_SUCCESS;
}

int main() {
    //setlocale(LC_ALL, "Russian");
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    std::cout << "МЕНЮ РЕШЕНИЯ СЛУ МЕТОДОМ ГАУССА:\n";
    std::cout << "1 - Получить данные для построения графиков сравнения\n";
    std::cout << "2 - Сравнить метод Гаусса с библиотекой Eigen (N на N)\n";
    std::cout << "3 - СТРЕСС-ТЕСТ: матрицы большого размера (1000 и 10000)\n";
    std::cout << "4 - СПЕЦ. МАТРИЦЫ: Тестирование на матрице Гильберта\n";

    int choice;
    std::cin >> choice;
    switch (choice)
    {
    case 1: return getData();
    case 2: return CompareGaussianAndLU();
    case 3: return TestLargeMatrices();
    case 4: return TestHilbertMatrix();
    default:
        std::cout << "Неверный выбор.\n";
        return 0;
    }
}
```

Файл matrixlib.hpp

```
#ifndef MATRIXLIB_HPP

#define MATRIXLIB_HPP


#include <limits>

#include <string>

#include <vector>

#include <iostream>

#include <fstream>

#include <iomanip>

#include <random>
```

```
#include <type_traits>

#define STANDART_MATRIX_SIZE 5
#define MAXIMUM_ELEMENT_VALUE 10
#ifndef EPSILON
#define EPSILON 1e-50
#endif

template <class T>
class squareMatrix {
private:
    std::vector<std::vector<T>> M;
    int N;
    bool symmetric;
    bool singular;
public:
    squareMatrix(int matrixSide, bool isHilbert = false);
    squareMatrix();

    int getN() const { return this->N; }
    const T& getElement(int rowIndex, int columnIndex) const { return this-
>M[rowIndex][columnIndex]; }
    void setElement(int rowIndex, int columnIndex, T value) { this-
>M[rowIndex][columnIndex] = value; }

    T getRandomNumber();
    T getGilbertRandom(int i, int j);

    int getRowOfMaxElement(int colomn, int startRow) const;
    void swapRows(int rowI, int rowJ);
    std::vector<T>& getRow(int index) { return this->M[index]; }
};

template <class T>
squareMatrix<T>::squareMatrix(int matrixSide, bool isHilbert) : N(matrixSide) {
    M.resize(N, std::vector<T>(N));
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (isHilbert) {
                M[i][j] = getGilbertRandom(i, j);
            } else {
                M[i][j] = getRandomNumber();
```

```
            }
        }
    }
    this->symmetric = false;
    this->singular = false;
}

template <class T>
squareMatrix<T>::squareMatrix() : squareMatrix(STANDART_MATRIX_SIZE, false) {}

template <class T>
T squareMatrix<T>::getGilbertRandom(int i, int j) {
    // Формула элемента матрицы Гильберта H(i,j) = 1 / (i + j + 1)
    return static_cast<T>(1.0 / (i + j + 1));
}

template <class T>
T squareMatrix<T>::getRandomNumber() {
    if constexpr (std::is_same_v<T, int>) {
        static std::random_device rd;
        static std::mt19937 gen(rd());
        static std::uniform_int_distribution<int> dist(-MAXIMUM_ELEMENT_VALUE,
MAXIMUM_ELEMENT_VALUE);
        return dist(gen);
    }
    else if constexpr (std::is_same_v<T, double> || std::is_same_v<T, float> ||
std::is_same_v<T, long double>) {
        static std::random_device rd;
        static std::mt19937 gen(rd());
        static std::uniform_real_distribution<long double> dist(-
MAXIMUM_ELEMENT_VALUE, MAXIMUM_ELEMENT_VALUE);
        return static_cast<T>(dist(gen));
    }
    return static_cast<T>(0);
}

template <class T>
int squareMatrix<T>::getRowOfMaxElement(int colomn, int startRow) const {
    int targetRow = -1;
    T maxElem = -1.0;
    for(int i = startRow; i < N; ++i) {
        const T& currentElement = std::fabs(M[i][colomn]);
```

```
        if (currentElement > maxElem) {
            targetRow = i;
            maxElem = currentElement;
        }
    }
    return targetRow;
}

template <class T>
void squareMatrix<T>::swapRows(int rowI, int rowJ) {
    if(rowI < N && rowJ < N && rowI >= 0 && rowJ >= 0) {
        std::swap(M[rowI], M[rowJ]);
    }
}

#endif // MATRIXLIB_HPP
```